blocpress Product Backlog

Inhaltsverzeichnis

1. Epic 1: Document Rendering Engine . . . 2
2. Epic 2: Template Storage und Persistence . . . 3
3. Epic 3: Template-Entwicklung (blocpress-workbench) . . . 4
4. Epic 4: Pruefung und Freigabe (blocpress-proof) . . . 5
5. Epic 5: Administration (blocpress-admin) . . . 6
6. Epic 6: Portal und Micro-Frontends (blocpress-studio) . . . 7
7. Technische Schulden (Implementierung TI-2) . . . 8
8. Abhaengigkeiten . . . 9

Dieses Dokument bildet das Product Backlog von blocpress ab. Es leitet sich aus der Architekturdokumentation und dem Element Design Concept ab.

Legende:

Status	Bedeutung
✅ DONE	Implementiert und getestet
▶ IN PROGRESS	In Bearbeitung
○ OFFEN	Noch nicht begonnen

1. Epic 1: Document Rendering Engine

Module: blocpress-core, blocpress-render

Kern-Engine zur Dokumentengenerierung aus ODT-Templates und JSON-Daten. Einzige öffentlich exponierte API des Systems.

Ref	Story	Prio	Status
TF-5	Merge-Pipeline (Text-Blöcke, Bedingungen, Schleifen, Feldersetzung)	1	✅ DONE
TI-3	LibreOffice headless Export (ODT → PDF/RTF)	1	✅ DONE
TI-1	REST-API: `POST /render/template/upload` (Multipart)	1	✅ DONE
TI-1	REST-API: `POST /render/template` (JSON/Base64)	1	✅ DONE
TI-4	JWT-Authentifizierung (konfigurierbar via ENV)	1	✅ DONE
—	Swagger UI (`/q/swagger-ui`)	2	✅ DONE
—	Docker Image mit Healthcheck	2	✅ DONE
—	Quickstart-Samples und Landing Page	3	✅ DONE
UC-10	Template-ID-basiertes Rendering (`POST /render/{id}`)	2	✅ DONE
UC-10.1	Template-Versionierung mit validFrom-Datum	2	✅ DONE
—	Asynchrone Generierung ueber Message Queue	3	○ OFFEN

2. Epic 2: Template Storage und Persistence

Module: blocpress-render, blocpress-core

Datenbankanbindung und Storage Abstraction Layer als Grundlage fuer Template-Verwaltung und Stufenuebergabe.

Voraussetzung fuer Epics 3-6.

TI-2 (Datenbankinfrastruktur) ist implementiert. StorageService-Abstraktion und formale Repository-Schicht sind noch offen (werden inkrementell gebaut).

Ref	Story	Prio	Status
TI-2	Zwei PostgreSQL-Datenbanken (workbench, production) mit automatischem Template-Deploy via REST	1	DONE
—	Storage Abstraction Layer (`StorageService` Interface)	1	OFFEN
—	Entity E-1 (Template) und E-3 (generiertes Dokument)	1	OFFEN
—	Repository Layer fuer Schema `production` in blocpress-render	1	OFFEN

3. Epic 3: Template-Entwicklung (blocpress-workbench)

Modul: blocpress-workbench

Arbeitsumgebung fuer Template-Designer: Upload, Validierung, Bearbeitung, Bausteinverwaltung und Content Search.

Ref	Story	Prio	Status
UC-1	Template hochladen und validieren	1	✅ DONE
TF-1	Template-Validierung (User-Fields, Wiederholungsgruppen, Bedingungen)	1	✅ DONE
UC-3	Template-Details anzeigen	2	✅ DONE
UC-2	Template zur Freigabe einreichen und Workflow-Status aendern (workbench intern, Stufenuebergabe workbench → production bei APPROVED)	2	✅ DONE
UC-5	Template Management Dashboard mit Status-Filtern und Workflow	2	✅ DONE
UC-20	Testdaten-Generator aus Template-Feldern	2	✅ DONE
UC-21	Mehrere Test-Datensaetze pro Template verwalten	2	✅ DONE
TF-8	PDF-Speicherung als "expected result" fuer Regression-Tests	2	✅ DONE
UC-11	Regression-Test durchfuehren (Ist-PDF vs. expected PDF)	3	○ OFFEN
—	Bausteinverwaltung (Entity E-2): Upload, Workflow (DRAFT→SUBMITTED→APPROVED), Tab-Navigation Templates/Bausteine, kombinierte Statusansicht (APPROVED + Draft pro Name)	2	✅ DONE
—	WebDAV-Endpoint (`/api/webdav/`) für Baustein- und Template-Zugriff aus LibreOffice (GET/PUT/PROPFIND, DRAFT read-write, APPROVED read-only unter `/released/`, GNOME Files / davfs2 kompatibel)	2	✅ DONE
UC-19	Content Search (Volltextsuche ueber alle Templates)	3	○ OFFEN
TI-7	Elasticsearch-Integration	3	○ OFFEN

4. Epic 4: Pruefung und Freigabe (blocpress-proof)

Modul: blocpress-proof

Workflow-gesteuerte Qualitaetssicherung mit Reviewer-Freigabe, Regressionstests und Compliance-Reviews. (Vier-Augen-Prinzip organisatorisch durchgesetzt.)

> Interim-Implementierung (TI-2): Die Freigabe (UC-8, TF-2) findet derzeit im Rahmen von blocpress-workbench statt — Template-Status wird via `PUT /api/workbench/templates/{id}/status` auf APPROVED gesetzt, was automatisch das Template in die production-Datenbank deployt. Ein separates blocpress-proof Modul existiert noch nicht. Die Stories dieses Epics beschreiben den Zielzustand.

Ref	Story	Prio	Status
UC-8	Template freigeben (Reviewer-Freigabe, organisatorisch durchgesetzt)	1	○ OFFEN
TF-2	Workflow-Status aendern (Reviewer muss Freigabe erteilen)	1	○ OFFEN
—	Stufenuebergabe proof → production (Kopie)	1	○ OFFEN
UC-14	Test Case erstellen (Baseline-PDF)	2	○ OFFEN
UC-16	Regressionstest ausfuehren	2	○ OFFEN
TF-6	PDF-Vergleich (Baseline vs. aktuell)	2	○ OFFEN
UC-12	Compliance-Review durchfuehren	3	○ OFFEN
TF-4	Review-Datum berechnen (1/3/5 Jahre)	3	○ OFFEN
TF-7	Automatische Compliance-Review-Ueberwachung (Scheduler)	3	○ OFFEN
TI-5	Message Queue fuer asynchrone Regressionstests	3	○ OFFEN

5. Epic 5: Administration (blocpress-admin)

Modul: blocpress-admin

Benutzerverwaltung, Rollenzuweisung und Audit-Logging.

Ref	Story	Prio	Status
UC-22	Benutzerverwaltung (CRUD, Rollenzuweisung)	1	○ OFFEN
—	Entity E-9 (Users) und E-10 (Audit-Logs)	1	○ OFFEN
—	RBAC-Pruefung bei jedem API-Call	2	○ OFFEN
—	Audit-Logging aller Workflow-Aenderungen	2	○ OFFEN

6. Epic 6: Portal und Micro-Frontends (blocpress-studio)

Modul: blocpress-studio

Portal-Shell als zentraler Einstiegspunkt mit Micro-Frontend-Integration der SCS-Module.

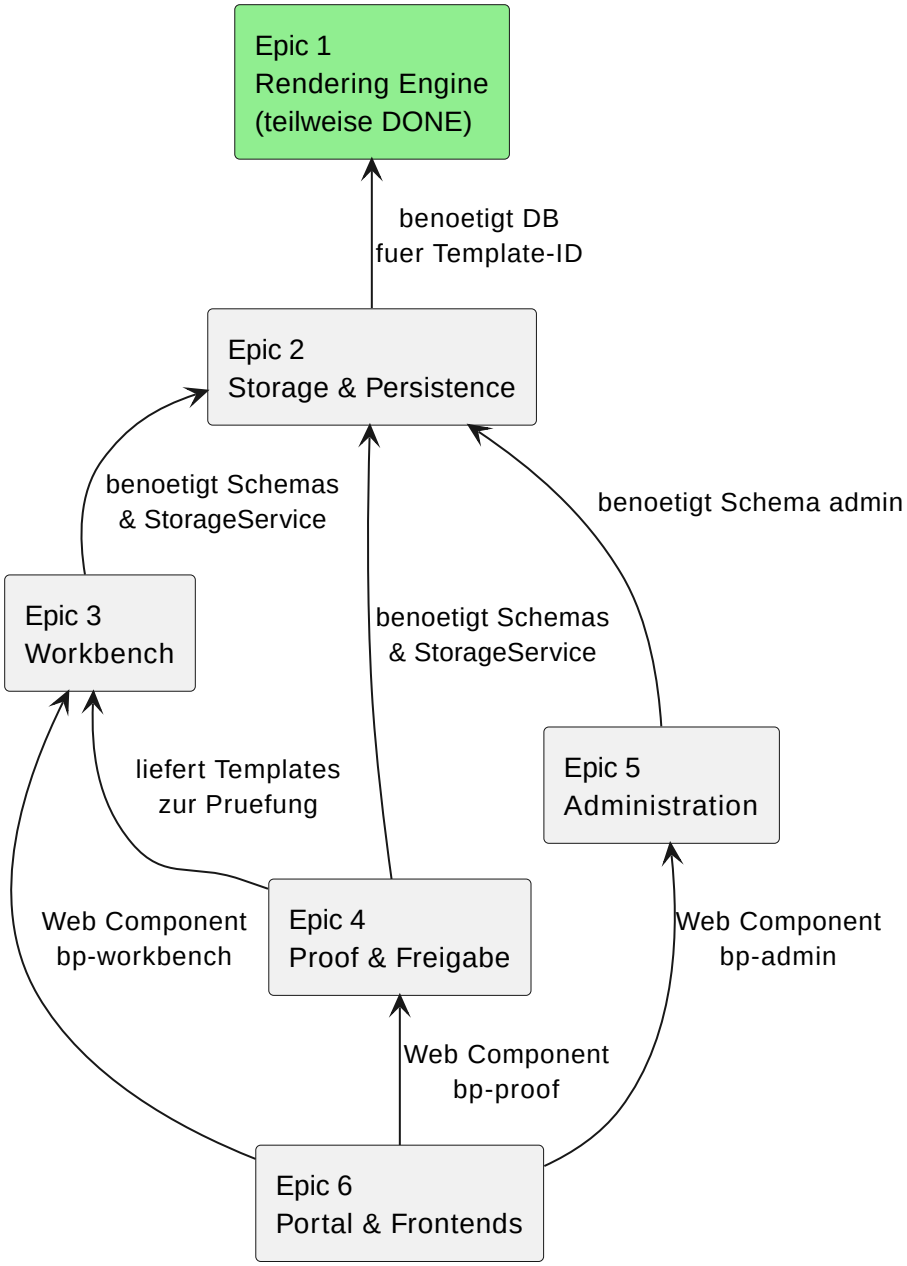
Ref	Story	Prio	Status
—	Portal-Shell (Navigation, JWT-Handling)	1	✅ DONE
—	Import Maps fuer Web Components	1	✅ DONE
—	Web Components: `<bp-workbench>`, `<bp-proof>`, `<bp-admin>`	2	▶ IN PROGRESS
—	JWT-Weitergabe als Property an Web Components	2	✅ DONE

7. Technische Schulden (Implementierung TI-2)

Folgende Probleme wurden bei der Code-Review der TI-2-Implementierung identifiziert:

ID	Problem	Kritikalitaet	Behebung
TD-8	**RenderImportClient URL-Doppelpfad**: `application.properties` konfiguriert Basis-URL als `${RENDER_URL}/api`, waehrend `@Path("/api/render/templates/import")` erneut `/api` enthaelt. Effektive URL: `…/api/api/render/…` → 404.	KRITISCH	`@Path` in `RenderImportClient.java` auf `/render/templates/import` aendern (ohne `/api` -Praefix)
TD-9	**renderDocumentById/ByName ohne JAX-RS-Annotationen**: Methoden in `RenderResource.java` haben keine `@POST`, `@Path`, `@Produces`, `@Consumes` Annotationen und sind nicht als HTTP-Endpoints exponiert.	KRITISCH	JAX-RS-Annotationen hinzufuegen oder in die OpenAPI-generierte Schnittstelle integrieren
TD-10	**Cache-Invalidierung fehlt**: `TemplateCache` verwendet `@CacheResult` (10 min TTL), aber `TemplateImportResource` hat kein `@CacheInvalidate`. Bei Re-Import eines Templates wird veralteter Inhalt bis zu 10 Minuten ausgeliefert.	HOCH	`@CacheInvalidate(cacheName="templates")` in `TemplateImportResource.importTemplate()` hinzufuegen
TD-11	**Kein Timeout fuer RenderImportClient**: `RenderImportClient` hat kein konfigurierbares Timeout. Bei nicht erreichbarem render-Service blockiert der HTTP-Client unbegrenzt.	HOCH	MicroProfile Timeout-Annotation oder `quarkus.rest-client."render-import".connect-timeout=5000` konfigurieren
TD-12	**Case-Sensitivity-Bug bei OutputType**: `outputTypeStr` wird gegen Kleinbuchstaben ("pdf", "rtf", "odt") verglichen, aber `enum.toString()` liefert ggf. Grossbuchstaben (abhaengig von Enum-Definition).	HOCH	`outputTypeStr.toLowerCase()` oder `equalsIgnoreCase()` verwenden
TD-13	**Falscher Default-JDBC-URL in render**: `application.properties` verwendet `localhost:5432/workbench` als Default, sollte `localhost:5432/production` sein.	MITTEL	Default in `blocpress-render/src/main/resources/application.properties` auf `jdbc:postgresql://localhost:5432/production` aendern

8. Abhaengigkeiten



Reihenfolge:

1. **Epic 6** (Studio) — Minimale Portal-Shell als Einstiegspunkt (Navigation, JWT)
2. **Epic 3** (Workbench) — Erster fachlicher Anwendungsfall fuer Template-Designer (UC-1: Upload)
3. **Epic 1** (Rendering Engine) — Offene Punkte (Template-ID-Rendering, Caching) nach Bedarf
4. **Epic 4** (Proof & Freigabe) — Aufbauend auf Workbench
5. **Epic 5** (Administration) — RBAC und Audit-Logging
6. *Epic 2 (Storage & Persistence) entsteht inkrementell beim Aufbau der Epics 3, 4, 5*